INTRODUÇÃO AO MATLAB

1- As atividades de laboratório deverão ser feitas individualmente, e os relatórios entregues no prazo máximo 15 dias após a realização do Experimento;
2- As soluções deverão ser clara, objetivas e organizadas. Caso haja figuras, tabelas e equações,essas deverão ser numeradas e referenciadas. **Não deverá ser utilizado no relatório material já apresentado na introdução.**

OBJETIVO

O laboratório tem o seguinte objetivo fixar a operação e o uso do Matlab e do Simulink, programas que serão usados no decorrer de todo o curso.

INTRODUÇÃO AO MATLAB

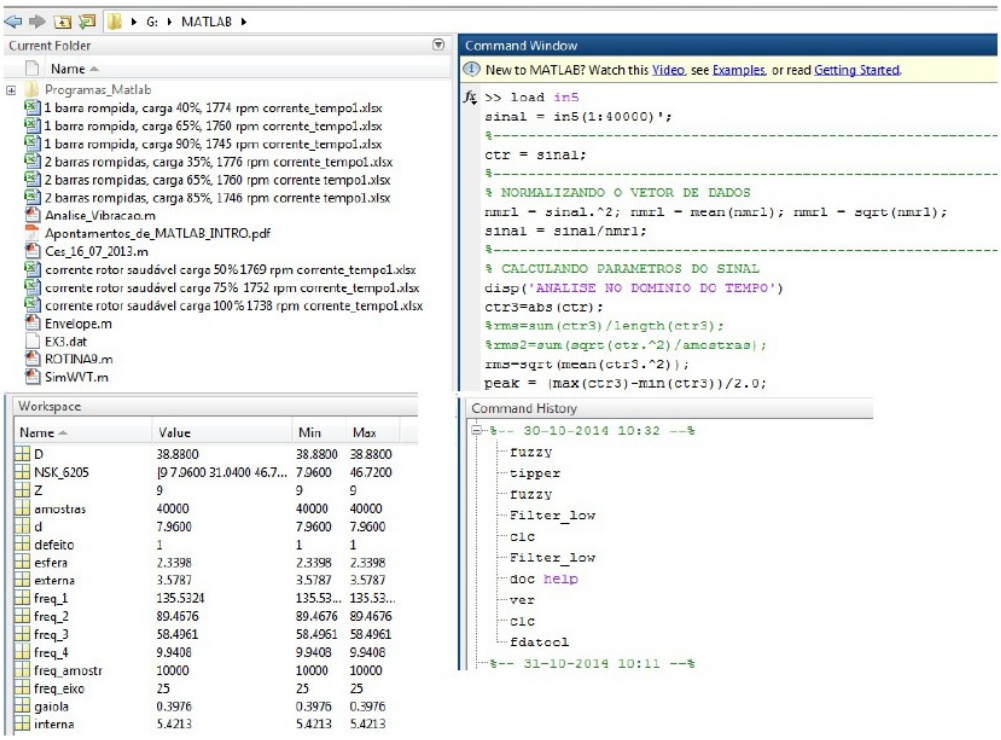
O MATLAB (de *MATrix LABoratory*) é um software produzido pela Mathworks, Inc., de modo genérico o MATLAB serve para tratar matrizes e números complexos da mesma forma que uma calculadora trata números reais. Além disso, o software possui recursos de programação, agindo como uma linguagem interpretada, semelhante a C ou Pascal, porém voltada para processamento numérico intensivo. Contendo programas para projeto de controle, recursos gráficos dentre outros.
A interface do MATLAB é composta basicamente de uma janela de comandos, com um ***prompt*** característico (>>). Além dessa janela de comandos, podem coexistir diversas janelas, que podem ser habilitadas ou desabilitada através do comando "**View**".

TUTORIAL DO MATLAB

Neste laboratório algumas demonstrações de comandos do MATLAB. Você deve digitar os comandos indicados para se familiarizar com o MATLAB.

Quando o MATLAB é carregado surge uma janela principal dividida em quatro sub-janelas, com diferentes informações, conforme mostradas na Figura 4.1. As sub-janelas podem ser ativadas/desativadas, através da opção Desktop.

a) Command Windows (Janela de Comandos): permite executar os comandos e funções no MATLAB;
b) Current Folder (Navegador de Diretórios): permite visualizar ou alterar o diretório de trabalho;
c) Workspace (Espaço de Dados): permite visualizar e manipular as variáveis existentes em memória, na sessão do MATLAB;
d) Command History (Histórico de Comandos): apresenta os últimos comandos executados.



Janela de comando

Nesta janela podem-se introduzir variáveis, comandos, funções e executar arquivos M.
Quando a janela está pronta para receber comandos, surge o símbolo >>, designado prompt.
Os comandos **demo** e **help** podem ser muito úteis para se ter uma idéia geral do funcionamento do Matlab (use os sempre que tiver dúvidas). O comando

>> **help xxx**

em que **xxx** representa uma função ou um comando específico, fornece informações apenas a respeito desse comando ou função. Uma vez feito isso, digite:

>> **a=[1 2;3 4]**

Para definir uma matriz **a** (note que maiúsculas e minúsculas fazem diferença). O MATLAB usa colchetes para definir matrizes e um ponto e vírgula como separador de linhas. Um espaço em branco ou uma vírgula podem ser usados para separar colunas. Note também que o MATLAB responde ao comando acima apresentando a matriz na janela de comandos.
Se o comando fosse terminado por um ponto e vírgula ("**;**"), a matriz seria definida da mesma forma, mas não seria apresentada na tela (isto é particularmente útil para matrizes muito grandes, cuja exibição pode preencher diversas telas).
Digite agora
>> **whos**
Para verificar quais são as variáveis já definidas. Defina agora uma outra matriz
>> **b=[5 6;7 8]**
Com isso, agora será possível realizar algumas operações matriciais simples:
>> **c=a*b** (multiplicação)
>> **d=inv(c')**

E algumas um pouco mais complexas:
>> **[f,g]=eig(c)** (fornece os autovetores de c em f e os autovalores em g)
>> **f=a(:,1)*b(2,:)** (produto da primeira coluna de a com a segunda linha de b)
A notação utilizada é bastante útil. O símbolo dois pontos (":") significa "todos os elementos". Assim quando se escreve

a(:,1), leia-se "todas as linhas de **a**, mas somente a primeira coluna". Obviamente, **a(:,1)** é uma matriz de dimensão 2x1.
Uma outra aplicação do Matlab, são as funções do Matlab que lidam com polinômios, as quais assumem que polinômios são descritos por vetores linha, em ordem decrescente de grau, isto é, o polinômio
$p(x) = 4x4 + 5x2 + x + 9$ é descrito como sendo o vetor **p = [4 0 5 1 9]**.
Operações sobre polinômios através das funções: **poly, roots, polyval, polyvalm, conv, deconv, residue, polyfit**
Um recurso muito útil no MATLAB é a geração e manipulação de vetores (vetores são matrizes com apenas uma linha ou uma coluna).
Um exemplo para isso é o calculo do espectro de potência de um sinal. Façamos um exemplo e aproveitemos para apresentar alguns recursos gráficos mais simples. O comando

>> **t=0:0.001:0.5;**

Define um vetor linha **t** contendo números de 0 a 0,5 com passo de 0,001, isto é [0 0,001 0,002 ...0,5].

>> **size(t);** fornece as dimensões do vetor t.
>> **x=sin(2*pi*50*t);**calcula o seno da expressão entre parênteses para cada elemento de **t** e armazena o resultado nas posições correspondentes de **x**. "pi" representa p=3,1415923...

Para traçar um gráfico de **x** contra **t**, digite:

>> **plot(t,x);**

O gráfico aparece em uma janela diferente, possivelmente **com** o titulo "*Figure 1*". Diversos gráficos podem ser exibidos simultaneamente. Para saber como, digite:

>> **help figure**

O comando plot deverá ser um dos comandos que você mais utilizará. Certifique-se de que você saiba operá-lo em detalhe. Para calcular a transformada rápida de Fourier de **x**, digite

>> **y=fft(x);**

>> **pyy=y.*conj(y);**

Note que o operador "**.***" (que é um outro operador, diferente do operador de multiplicação "*") faz multiplicação elemento a elemento, isto é, o primeiro elemento de um vetor multiplicado pelo primeiro elemento do outro, etc. O operador "*" realiza a multiplicação usual de matrizes.
Para se traçar um gráfico com o espectro de potência, é necessário inicialmente se definir uma abscissa adequada:

>> **f=1000*(0:255)/512;**
Digite então

>> **plot(f,pyy(1:256))**
E com isso serão colocados no gráfico somente os 256 primeiros elementos de **pyy**.

Exemplo: Gere um sinal senoidal, com frequência de 5 Hz, amplitude de 2 V por 3 segundos. Considere uma amostragem a uma taxa de 60 Hz. Calcule a transformada de Fourier desse sinal e esboce o seu gráfico.

Solução:

```
close all
clear
clc;
fs=60; t=0:1/fs:3;%Frequencia de amostragem e tempo de 3 segundos
N=length(t); %Tamanho do vetor tempo
f=5; sinal=2*sin(2*pi*f*t);% Gerando o sinal de 5 Hz
subplot (2,1,1);plot (t, sinal);
title ('Sinal Senoidal de 5 Hz (Dominio do Tempo)');
xlabel('Tempo (s)');ylabel ('Amplitude (V)');
y = fft(sinal);%Calcula a Transformada de Fourier do Sinal
y = y(1:floor(length(y)/2));
% Ajuste eixo defrequencia para o espectro
freq = (0:N-1)*fs/N;
freq = freq(1:floor(length (freq)/2));
subplot (2,1,2)
plot(freq, abs (y))
title ('Espectro do sinal (Dominio da Frequencia)')
ylabel('|y(f)|')
xlabel ('Frequencia (Hz)')
```

As funções acima, bem como muitas outras, são na verdade escritas na própria linguagem de programação do Matlab. A função **roots**, por exemplo, está no arquivo texto ...\MATLAB\TOOLBOX\MATLAB\POLYFUN\ROOTS.M.
Para ver seu conteúdo basta digitar

>> **type roots**

Esses arquivos, denominados *M-files*, podem ser de dois tipos: *functions* e *scripts. Scripts* são simplesmente seqüências de comandos, como acima, dispostos em um arquivo texto com extensão ".m". *Functions* são estruturas mais complexas, que apresentam certas características básicas, que são as seguintes:
i) Existe passagem de parâmetros entre o *workspace* do Matlab (*workspace* ou espaço de trabalho pode ser entendido a grosso modo como o conjunto de variáveis que pode ser observado com o comando whos na janela de comandos) e esta é a única troca de informações. Por exemplo, o comando **y=cos(x)** passa à função **cos** um argumento **x** e recebe como resposta **y**. Não há, além disso, nenhuma troca de informações.
ii) O ambiente de cada função é independente, isto é, uma função não reconhece variáveis definidas fora dela, e outras funções ou o *workspace* não reconhecem variáveis definidas nela. Programadores de Pascal perceberão uma analogia clara entre *scripts* e *functions* do Matlab com *procedures* e *functions* do Pascal, com a diferença de que no Matlab (pelo menos na versão considerada) não se pode declarar funções dentro de funções.

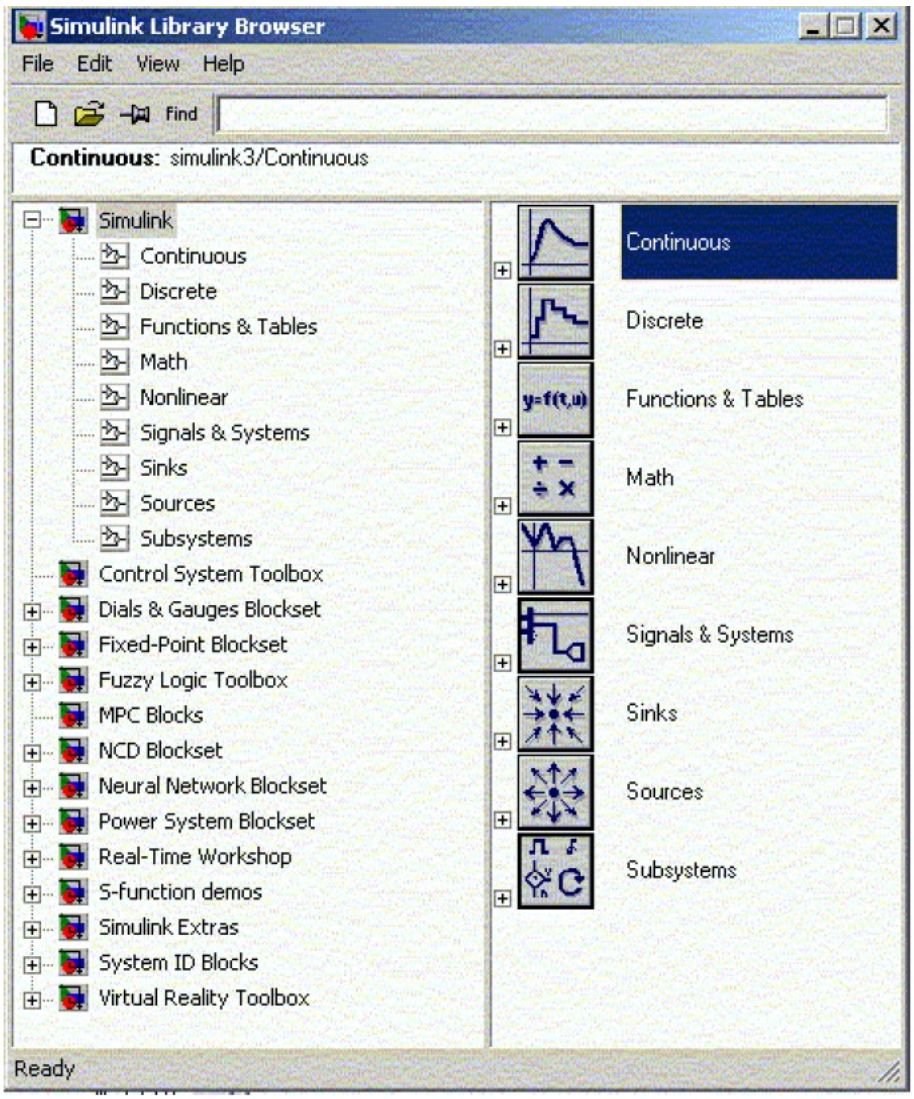
Você será capaz de escrever seus próprios *scripts e functions* para automatizar tarefas repetitivas eventualmente realizar algumas mais complexas. A programação no Matlab é muito semelhante a outras linguagens ,e se você estiver familiarizado com qualquer uma delas não terá problemas em fazer programas para o Matlab.

O SIMULINK

O Simulink, produzido pela *The Mathworks. Inc*., é um programa destinado à modelagem, simulação e análise de sistemas dinâmico. Ele opera em conjunto com o Matlab, mas ao contrário dele, a sua interface com o usuário é totalmente gráfica,feita a partir da interconexão de blocos. O simulink é acessado a partir do Matlab através do ícone , ou utilizando o comando:

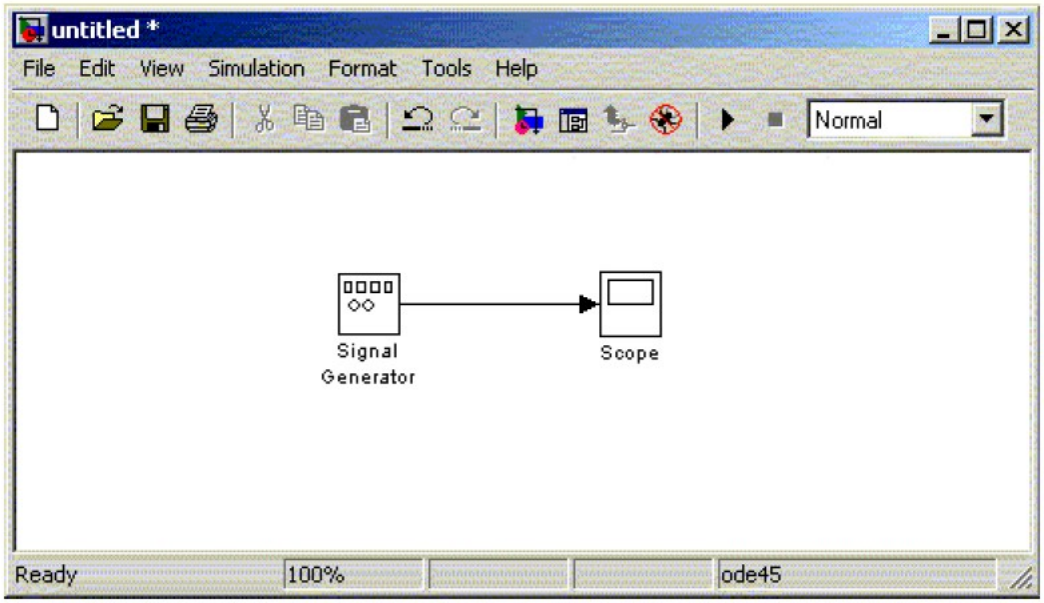
>> simulink

Cada uma das caixas (Sources, Sinks, etc.) pode ser acessada a partir de um *c1ick* do mouse, fazendo com que se abram novas janelas. Cada uma delas contém blocos que podem ser interligados, gerando um sistema a ser simulado.

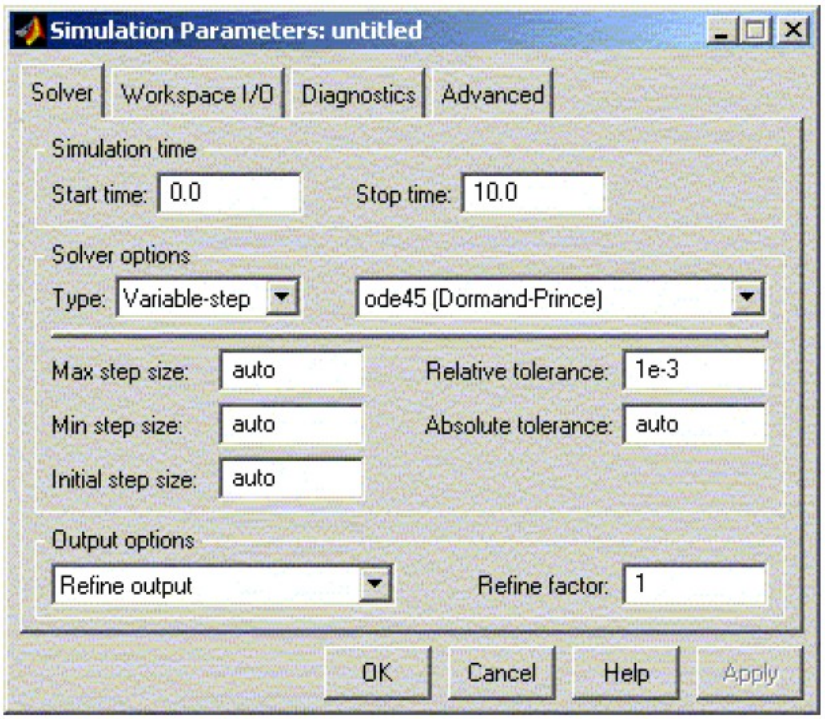


Breve Introdução Simulink

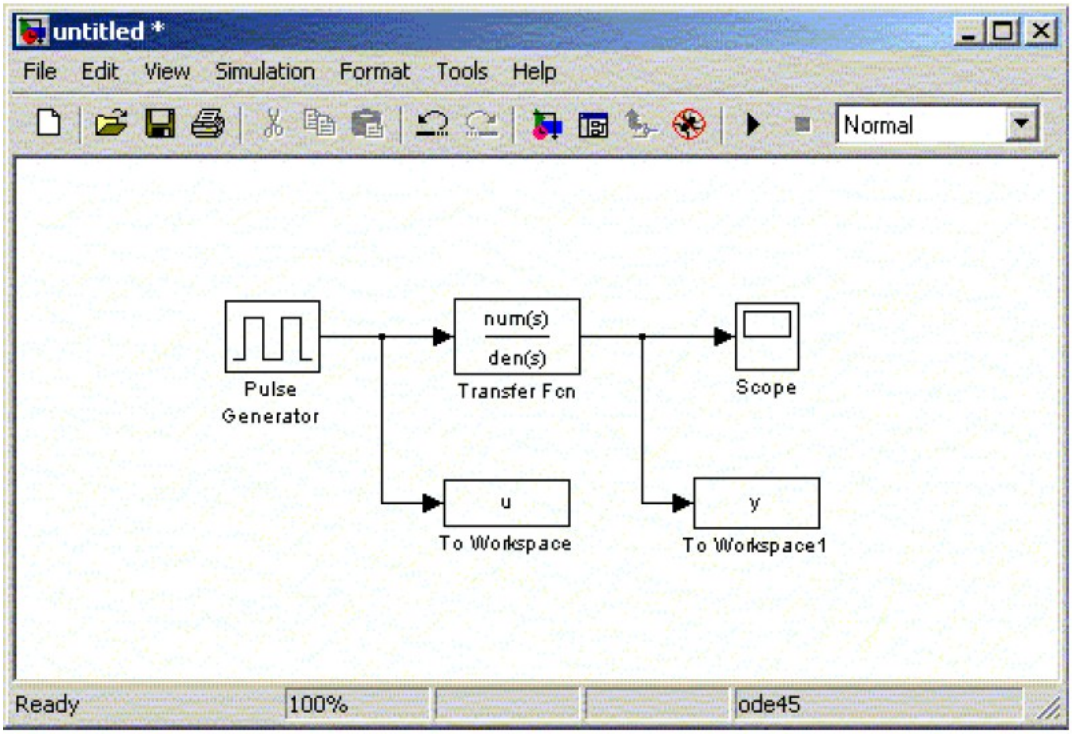
Para criar um novo sistema basta escolher FILE>NEW no menu da parte superior da janela. Com isso uma nova janela em branco, porém com a mesma barra de menus, aparecerá na tela.
Como primeira tarefa você deverá criar um diagrama como o descrito abaixo, copiando os blocos de seus respectivos grupos. Faça isso arrastando-os com o mouse. A conexão entre os blocos é feita da mesma forma.



Uma vez que você tenha feito isso, *double-click* em ambos os blocos para ter acesso às suas configurações.Configure o gerador de sinal para gerar uma onda dente de serra com uma freqüência de 1 Hz e com uma amplitude de pico unitária. Configure o osciloscópio para exibir pelo menos uns 10 períodos da onda.
Antes de se iniciar a simulação é necessário entender um pouco o seu funcionamento. O simulink, como todos os programas que fazem simulação de sistemas dinâmicos, se utiliza de algoritmos de integração para resolver a evolução temporal das equações do sistema numericamente. A janela abaixo, obtida do menu **SIMULATION>PARAMETERS** ilustra como o usuário pode fazer uso dos diferentes algoritmos e de suas características.



Finalmente, para se iniciar a simulação basta usar a opção **SIMULATION>START** do menu principal. Com a primeira tarefa cumprida, passe ao seguinte exercício.
Monte o diagrama abaixo. Note que você pode utilizar as variáveis do Matlab num e den (se você ainda as tiver)como argumentos do bloco de função de transferência. Via de regra, qualquer bloco do Simulink aceita variáveis do w*orkspace* do Matlab como parâmetros de entrada. Escolha a freqüência do gerador de pulsos de modo que o sistema saia de uma situação de regime e atinja novamente o regime entre transições na entrada (isto é, use um período suficientemente grande para que a resposta do sistema a cada pulso seja parecida com a resposta do sistema a um degrau). Faça a simulação para alguns períodos e verifique ao final, o conteúdo das variáveis u e y do Matlab (variáveis referidas nos blocos "*To Workspace*").



PARTE PRATICA

1.Simule no Matlab uma onda senoidal com as seguintes características
Amplitude = 5 V
Freqüência = 1 KHz
e plote o gráfico correspondente (utilize as funções xlabel, ylabel, title e grid)

2.Repita o exercício 1, simulando agora no Simulink e observe o sinal gerado através do osciloscópio.

3.Utilizando o resultado do exercício 1, verifique quantas amostras foram necessárias para representar o sinal gerado (utilize a função length)

4.Simule um sinal senoidal considerando somente 100 amostras para representá-lo. Considere as mesmas características do exercício 1. Apresente os resultados através de gráfico e tabela no prompt do Matlab.

5.Utilizando o Simulink, gere várias formas de onda (senoidal, quadrada) e observe estas ondas em um único osciloscópio. (Nota: Utilize o MUX para ver os dois sinais em um único osciloscópio).
OBS. Atente para o fato de que o argumento das funções senoidais no Matlab são sempre expressos em radianos.